

перейдем немного в другую плоскость. метеостанция работает на частоте 915 мгц. с определенной полосой пропускания. а вот вопрос, какая полоса пропускания оптимальная для работы с уличной погодной станцией. официальные какие характеристики у WS1080

Для семейства станций Fine Offset (к которым относится WS1080) официальные технические характеристики радиоканала и реальные требования к полосе пропускания (Bandwidth) выглядят следующим образом:

Официальные параметры радиоканала WS1080:

- **Символьная скорость (Data Rate):** 17.241 kbps.
- **Модуляция:** FSK (Frequency Shift Keying).
- **Девияция (Frequency Deviation):** $\pm 45\text{--}50$ кГц (в среднем 47.6 кГц).

Расчет минимально необходимой полосы (Правило Карсона):

Для качественного приема FSK сигнала минимальная полоса пропускания рассчитывается по формуле:

$$BW \approx (2 \times \text{Deviation}) + \text{Symbol Rate}$$

Подставляем значения: $(2 \times 50 \text{ кГц}) + 17.241 \text{ кГц} \approx 117 \text{ кГц}$.

Почему «минимальная» полоса не является «оптимальной»:

В идеальных условиях 117 кГц достаточно, но для реальной эксплуатации WS1080 на модуле CC1101 это приведет к потере данных. Вот почему:

1. **Дрейф частоты (Frequency Drift):** Уличные датчики WS1080 используют недорогие кварцевые резонаторы. При сильном морозе или жаре частота «уплывает» на $\pm 20\text{--}40$ кГц. Если ваша полоса будет узкой (например, 120 кГц), сигнал просто выйдет за пределы фильтра.
2. **Точность CC1101:** Кварц на китайских модулях CC1101 тоже имеет погрешность. Ошибка в 30 ppm на частоте 915 МГц дает смещение еще на ± 27 кГц.
3. **Итоговое смещение:** Суммарно датчик и приемник могут «не совпадать» на $50\text{--}70$ кГц.

Оптимальная полоса (RX Bandwidth) для CC1101:

На практике для Fine Offset (WH1080/WS1080) в библиотеках (типа `rtl_433` или `RadioLib`)

устанавливаются следующие значения:

- **270 кГц:** Рекомендуемый минимум. Позволяет игнорировать небольшие температурные отклонения.
- **325 кГц: Оптимальное значение.** Обеспечивает надежный прием даже при экстремальных температурах на улице и неточности кварца на CC1101, сохраняя при этом хорошую чувствительность.
- **400 кГц:** Максимально допустимая. Полезна, если датчик «кривой» или вы не можете точно попасть в частоту, но при этом возрастает уровень шума (Noise Floor), и дальность приема падает.

Резюме: Для вашей метеостанции установите **325 кГц**. Это позволит «проглатывать» уход частоты датчика без потери пакетов.

Хотите узнать, как проверить, насколько сильно «уплыла» частота вашего датчика относительно настройки 915.00 МГц?

да, было бы здорово придумать что-то типа спектроанализатора на CC1101. если конечно это реализуемо.

Это абсолютно реализуемо. Хотя CC1101 не является полноценным SDR-приемником, у него есть возможность быстрого сканирования RSSI (уровня сигнала) на разных частотах. Мы можем написать функцию, которая «пробежится» по диапазону вокруг 915 МГц с мелким шагом и выведет график в консоль.

Поскольку уличная станция передает пакет всего **~100-150 мс** раз в **48 секунд**, поймать этот момент сканером сложно. Но мы можем запустить «быстрый сканер», который будет непрерывно мониторить эфир.

Код «Спектроанализатора» для вашего скетча

Добавьте эту функцию в ваш скетч. Она сканирует диапазон ± 250 кГц от центральной частоты.

```
void runSpectrumAnalyzer() {  
    Serial.println(F("\n--- SCANNING AROUND 915 MHz ---"));  
    float start_f = current_freq - 0.25; // -250 кГц  
    float end_f = current_freq + 0.25;    // +250 кГц  
    float step = 0.01; // Шаг 10 кГц  
  
    for (float f = start_f; f <= end_f; f += step) {  
        radio.setFrequency(f);  
        delay(10); // Даем время чипу перестроиться и измерить RSSI  
        float rssi = radio.getRSSI();  
    }  
}
```

```

// Визуализация в Serial
Serial.printf("%3.2f MHz [%4.1f]: ", f, rssi);
int barLen = map(constrain(rssi, -115, -30), -115, -30, 0, 50);
for (int i = 0; i < barLen; i++) Serial.print('=');
Serial.println();
}
radio.setFrequency(current_freq); // Возвращаемся на базу
}

```

Как это использовать для калибровки WS1080:

1. **Поймите момент:** Так как пакет короткий, вам нужно запустить сканирование в тот момент, когда вы знаете, что датчик начал передачу (ориентируйтесь по морганию светодиода на самом уличном блоке или по логам `rtl_433_ESP`, если они идут).
2. **Ищите «Горб»:** На частоте передачи RSSI подскочит с -108 до -50...-60 дБм.
3. **Определите центр:** Если вы видите, что максимальный уровень сигнала не на 915.00, а, например, на 915.03 — значит, кварц датчика уплыл на 30 кГц вверх.
4. **Сделайте выводы:**
 - Если сигнал узкий и четкий — полосы 270 кГц хватит.
 - Если сигнал «размазан» или прыгает — ставьте **325 кГц**.

Лайфхак для Arduino IDE:

В Arduino IDE есть инструмент «Плоттер по последовательному соединению» (Serial Plotter). Если вы будете выводить только число RSSI, вы увидите график в реальном времени.

Хотите, я перепишу функцию сканера так, чтобы она работала автоматически: «слушала» эфир и при всплеске сигнала сама находила его точный центр?

Код для вызова функции через API:

```

server.on("/api/scan", HTTP_GET, []() {
  runSpectrumAnalyzer();
  server.send(200, "text/plain", "Scan finished. Check Serial Monitor.");
});

```

Это поможет вам точно узнать, на какой частоте на самом деле «кричит» ваша погодная станция.

да, это очень полезная функция. и еще, конечно-же, погодная станция передает раз в

48 секунд, но этих секунд много... было-бы замечательно реализовать такой сканер, который бы смещался автоматически в низ или вверх по частоте, смотрел уровень сигнала, запоминал его и двигался бы дальше на следующий шаг. а то, что он будет работать очень медленно, ну так и мы не торопимся....

Идея отличная. Чтобы «поймать» передатчик, который работает всего 150 мс раз в 48 секунд, обычное быстрое сканирование бесполезно — мы просто проскочим момент передачи.

Нам нужен «**Статистический сканер**». Логика такая: мы встаем на частоту (например, 914.90) и ждем на ней чуть больше 48 секунд (например, 50 секунд). Если за это время зафиксирован пик RSSI выше порога — запоминаем его. Потом переходим на 914.91 и повторяем.

Это займет время (10 шагов = 8 минут), но зато мы получим **реальный спектр** вашего датчика.

Реализация статистического сканера

Добавьте этот код в ваш скетч. Я сделал его «неблокирующим», чтобы веб-страница и остальной функционал продолжали работать.

```
// Переменные для сканера
bool scan_active = false;
float scan_current_freq = 914.80; // Начало диапазона
float scan_best_freq = 915.00;
float scan_max_rssi = -150.0;
unsigned long scan_step_start = 0;
const unsigned long SCAN_STEP_TIME = 50000; // 50 секунд на шаг

void updateAutoScanner() {
    if (!scan_active) return;

    // Инициализация шага
    if (scan_step_start == 0) {
        scan_step_start = millis();
        radio.setFrequency(scan_current_freq);
        LOG("SCAN", "Testing freq: %.3f MHz...", scan_current_freq);
    }

    // Постоянно мониторим пик на текущем шаге
    float current_rssi = radio.getRSSI();
```

```

    if (current_rssi > scan_max_rssi) {
        scan_max_rssi = current_rssi;
    }

    // Если время шага вышло – переходим дальше
    if (millis() - scan_step_start > SCAN_STEP_TIME) {
        LOG("SCAN", "Result for %.3f MHz: Max RSSI = %.1f dBm", scan_current_freq,
scan_max_rssi);

        // Тут можно сохранять данные в массив для вывода графика

        // Переход к следующему шагу
        scan_current_freq += 0.02; // Шаг 20 кГц
        scan_step_start = 0;
        scan_max_rssi = -150.0;

        // Конец диапазона (например, сканируем до 915.20)
        if (scan_current_freq > 915.20) {
            scan_active = false;
            radio.setFrequency(current_freq); // Возвращаемся на рабочую
            LOG("SCAN", "Scanning finished.");
        }
    }
}

```

Как это интегрировать:

1. В `loop()`: добавьте вызов `updateAutoScanner()`;
2. В `server.on`: добавьте команду для запуска:

```

server.on("/api/start_scan", HTTP_GET, []() {
    scan_active = true;
    scan_current_freq = 914.80;
    scan_step_start = 0;
    server.send(200, "text/plain", "Scanner started. Watch Serial log.");
});

```

Как читать результат:

Когда сканер закончит работу, посмотрите в лог. Частота, на которой `Max RSSI` был самым высоким (например, -45 dBm против фоновых -108 dBm), и есть **истинная центральная частота** вашего датчика.

Почему это важно для WS1080?

Если вы обнаружите, что пик на частоте 915.08, а у вас настроено 915.00, то даже с полосой 325 кГц вы будете принимать сигнал «краем», что уменьшит дальность. Подстройка под реальный пик позволит вам принимать станцию даже через несколько стен.

Добавить ли в этот сканер автоматическое сохранение «лучшей» найденной частоты в настройки?

да

Это отличное решение. Мы превратим сканер в «интеллектуального помощника», который не просто найдет сигнал, но и сам настроит станцию на максимальную чувствительность.

Я доработал логику: теперь сканер сравнивает результаты каждого шага и в конце выбирает частоту с самым сильным сигналом.

Доработки для вашего скетча

1. Переменные (добавьте в блок объявлений)

```
bool scan_active = false;
float scan_current_freq = 914.80;
float scan_best_freq = 915.00;
float scan_best_rssi = -150.0;
float scan_step_max_rssi = -150.0;
unsigned long scan_step_start = 0;
const unsigned long SCAN_STEP_TIME = 52000; // 52 секунды (с запасом на цикл датчика)
```

2. Функция сканера (добавьте перед loop)

```
void updateAutoScanner() {
    if (!scan_active) return;

    // Начало нового шага
    if (scan_step_start == 0) {
        scan_step_start = millis();
        radio.setFrequency(scan_current_freq);
        LOG("SCAN", "Step [%.2f MHz] - Waiting for packet...", scan_current_freq);
    }

    // Замеряем пик на текущей частоте
    float current_rssi = radio.getRSSI();
    if (current_rssi > scan_step_max_rssi) {
```

```

        scan_step_max_rssi = current_rssi;
    }

    // Когда время шага вышло
    if (millis() - scan_step_start > SCAN_STEP_TIME) {
        LOG("SCAN", "Freq %.2f MHz: Max RSSI = %.1f dBm", scan_current_freq,
scan_step_max_rssi);

        // Проверяем, лучше ли это того, что мы видели раньше
        if (scan_step_max_rssi > scan_best_rssi) {
            scan_best_rssi = scan_step_max_rssi;
            scan_best_freq = scan_current_freq;
            LOG("SCAN", "NEW BEST: %.2f MHz (%.1f dBm)", scan_best_freq,
scan_best_rssi);
        }

        // Переходим к следующему шагу
        scan_current_freq += 0.02; // Шаг 20 кГц оптимален для поиска пика
        scan_step_start = 0;
        scan_step_max_rssi = -150.0;

        // Конец диапазона
        if (scan_current_freq > 915.20) {
            scan_active = false;

            // ЕСЛИ МЫ НАШЛИ РЕАЛЬНЫЙ СИГНАЛ (выше уровня шума)
            if (scan_best_rssi > -90.0) {
                current_freq = scan_best_freq;
                radio.setFrequency(current_freq);
                saveAll(); // Сохраняем в Preferences
                LOG("SCAN", "COMPLETE! Best frequency found and SAVED: %.2f MHz",
current_freq);
            } else {
                radio.setFrequency(current_freq); // Возвращаемся к старой
                LOG("SCAN", "FINISHED. No strong signal found. Reverting to %.2f MHz",
current_freq);
            }
        }
    }
}

```

3. Интеграция в `loop` и API

Не забудьте вызвать `updateAutoScanner()`; в основном цикле `loop()`.

Команда для запуска через браузер:

```
server.on("/api/start_scan", HTTP_GET, []() {
  scan_active = true;
  scan_current_freq = 914.80; // Начало диапазона
  scan_best_rssi = -150.0;
  scan_step_start = 0;
  server.send(200, "text/plain", "Autoscan started. Check serial for progress (takes ~17 min).");
});
```

Как это будет работать на практике:

1. Вы запускаете сканер. Скетч переходит на 914.80 МГц.
2. Он "сидит" на ней 52 секунды. Если датчик "крикнул", скетч запишет максимальную громкость (-40...-60 дБм).
3. Так он пройдет весь диапазон.
4. В конце он автоматически выберет частоту, на которой датчик был слышен громче всего, установит её как рабочую и запишет в энергонезависимую память (Preferences).

Совет: Во время сканирования положите датчик на расстоянии 3–5 метров. Если он будет слишком близко, RSSI везде будет "зашкаливать" (-15...-20 дБм), и сканер не сможет найти точный пик.

Вам помочь с выводом **статуса сканирования на саму веб-страницу**, чтобы видеть прогресс прямо в браузере?

да, и совсем здорово если нарисовать в веб интерфейсе АЧХ сигнала.

Это превратит вашу погодную станцию в настоящий диагностический прибор! Для реализации графической АЧХ (амплитудно-частотной характеристики) нам нужно:

1. **Массив данных:** хранить результаты каждого шага сканирования.
2. **API:** отдавать этот массив в формате JSON.
3. **Frontend:** нарисовать график (используем простую HTML5 Canvas, чтобы не тянуть тяжелые библиотеки).

1. Дополнения в серверную часть (C++)

Добавим массив для хранения результатов и обновим логику сканера:

```
// В блок переменных
float scan_results[21]; // Для диапазона 914.80 - 915.20 с шагом 0.02 (21 точка)
int scan_total_steps = 21;
int scan_current_step = 0;

void updateAutoScanner() {
    if (!scan_active) return;

    if (scan_step_start == 0) {
        scan_step_start = millis();
        radio.setFrequency(scan_current_freq);
    }

    float current_rssi = radio.getRSSI();
    if (current_rssi > scan_step_max_rssi) scan_step_max_rssi = current_rssi;

    if (millis() - scan_step_start > SCAN_STEP_TIME) {
        // Сохраняем результат для графика
        if (scan_current_step < scan_total_steps) {
            scan_results[scan_current_step] = scan_step_max_rssi;
            scan_current_step++;
        }

        if (scan_step_max_rssi > scan_best_rssi) {
            scan_best_rssi = scan_step_max_rssi;
            scan_best_freq = scan_current_freq;
        }

        scan_current_freq += 0.02;
        scan_step_start = 0;
        scan_step_max_rssi = -150.0;

        if (scan_current_freq > 915.20) {
            scan_active = false;
            if (scan_best_rssi > -95.0) {
                current_freq = scan_best_freq;
                radio.setFrequency(current_freq);
                saveAll();
            }
        }
    }
}
```

```

    }
  }
}

```

2. Новый API для фронтенда

Добавьте это в `setup()` в секцию обработчиков `server.on`:

```

server.on("/api/scan_data", HTTP_GET, []() {
  JsonDocument doc;
  doc["active"] = scan_active;
  doc["progress"] = scan_active ? (scan_current_step * 100 / scan_total_steps) : 100;
  JsonArray data = doc.createNestedArray("rssi");
  for (int i = 0; i < scan_total_steps; i++) data.add(scan_results[i]);
  String out;
  serializeJson(doc, out);
  server.send(200, "application/json", out);
});

```

3. Веб-интерфейс (HTML/JS)

Вставьте этот кусок в ваш `index.html`. Он создаст холст и будет рисовать на нем график:

```

<div id="scan-section" style="margin-top:20px; border:1px solid #ccc; padding:10px;">
  <h3>АЧХ радиоканала (914.80 - 915.20 MHz)</h3>
  <canvas id="scanChart" width="400" height="150" style="background:#f9f9f9;
width:100%;"></canvas>
  <button onclick="startScan()">Запустить сканирование</button>
  <span id="scanStatus"></span>
</div>

<script>
function startScan() {
  fetch('/api/start_scan').then(() => {
    alert("Сканирование запущено! Это займет около 18 минут.");
    pollScan();
  });
}

function pollScan() {
  fetch('/api/scan_data').then(r => r.json()).then(data => {
    drawChart(data.rssi);
  });
}

```

```

        document.getElementById('scanStatus').innerText = data.active ? `Прогресс: $
{data.progress}%` : "Готово";
        if (data.active) setTimeout(pollScan, 5000);
    });
}

function drawChart(values) {
    const canvas = document.getElementById('scanChart');
    const ctx = canvas.getContext('2d');
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const stepX = canvas.width / (values.length - 1);
    ctx.beginPath();
    ctx.strokeStyle = '#2196F3';
    ctx.lineWidth = 2;

    values.forEach((v, i) => {
        // Маппим RSSI (-120...-30) в высоту холста
        const y = Math.max(0, Math.min(150, (v + 120) * (150 / 90)));
        const x = i * stepX;
        if (i === 0) ctx.moveTo(x, 150 - y);
        else ctx.lineTo(x, 150 - y);

        // Рисуем точки
        ctx.fillStyle = '#ff0000';
        ctx.fillRect(x-2, 150-y-2, 4, 4);
    });
    ctx.stroke();
}
</script>

```

Как это будет выглядеть:

- Когда вы нажмете кнопку, скетч начнет переключать частоты.
- Раз в 5 секунд браузер будет запрашивать данные.
- На графике начнет прорисовываться линия. Когда сканер дойдет до частоты датчика, вы увидите резкий **пик** (горб).
- Это визуально подтвердит: «Вот здесь мой датчик работает громче всего!».

Удобно ли вам будет добавить этот HTML-код в ваш файл `index.html` самостоятельно, или прислать готовую JS-функцию для вставки?

прислать готовую функцию для дальнейшей интеграции в код

Для интеграции функционала АЧХ-сканера в ваш проект, я подготовил два блока: **Back-end** (логика на C++ для ESP32) и **Front-end** (JavaScript для веб-страницы).

1. Back-end: Переменные и Логика (C++)

Добавьте эти переменные в начало скетча и вызовите `updateAutoScanner()` в `loop()`.

```
// --- Параметры сканера ---
bool scan_active = false;
float scan_results[21]; // Массив для точек (шаг 0.02 от 914.80 до 915.20)
int scan_current_step = 0;
float scan_best_freq = 915.00;
float scan_best_rssi = -150.0;
float scan_step_max_rssi = -150.0;
unsigned long scan_step_start = 0;
const unsigned long SCAN_STEP_TIME = 52000; // 52 сек (цикл датчика 48с + запас)

void updateAutoScanner() {
    if (!scan_active) return;

    if (scan_step_start == 0) {
        scan_step_start = millis();
        radio.setFrequency(914.80 + (scan_current_step * 0.02));
    }

    float current_rssi = radio.getRSSI();
    if (current_rssi > scan_step_max_rssi) scan_step_max_rssi = current_rssi;

    if (millis() - scan_step_start > SCAN_STEP_TIME) {
        // Сохраняем результат шага
        scan_results[scan_current_step] = scan_step_max_rssi;

        if (scan_step_max_rssi > scan_best_rssi) {
            scan_best_rssi = scan_step_max_rssi;
            scan_best_freq = 914.80 + (scan_current_step * 0.02);
        }

        scan_current_step++;
        scan_step_start = 0;
    }
}
```

```

        scan_step_max_rssi = -150.0;

        if (scan_current_step >= 21) {
            scan_active = false;
            if (scan_best_rssi > -95.0) {
                current_freq = scan_best_freq;
                radio.setFrequency(current_freq);
                saveAll(); // Сохраняем лучшую частоту
            }
        }
    }
}

// Добавьте в setup() в секцию server.on:
/*
server.on("/api/start_scan", HTTP_GET, [])() {
    scan_active = true;
    scan_current_step = 0;
    scan_best_rssi = -150.0;
    for(int i=0; i<21; i++) scan_results[i] = -120.0;
    server.send(200, "text/plain", "OK");
});

server.on("/api/scan_data", HTTP_GET, [])() {
    JsonDocument doc;
    doc["active"] = scan_active;
    doc["step"] = scan_current_step;
    JsonArray data = doc.createNestedArray("rssi");
    for (int i = 0; i < 21; i++) data.add(scan_results[i]);
    String out;
    serializeJson(doc, out);
    server.send(200, "application/json", out);
});
*/

```

2. Front-end: JavaScript функция (для index.html)

Эта функция создает интерфейс и рисует график. Она ожидает, что в HTML есть тег `<canvas id="spectrumCanvas"></canvas>`.

```

/**
 * Инициализация и запуск отрисовки АЧХ
 * Вызывать при нажатии на кнопку "Сканировать"

```

```

*/
function initSpectrumScanner() {
    const apiDataUrl = '/api/scan_data';
    const apiStartUrl = '/api/start_scan';
    const canvas = document.getElementById('spectrumCanvas');
    if (!canvas) return;
    const ctx = canvas.getContext('2d');

    // Запуск сканирования на контроллере
    fetch(apiStartUrl).then(() => {
        console.log("Scanner started");
        updateChart();
    });

    function updateChart() {
        fetch(apiDataUrl)
            .then(response => response.json())
            .then(data => {
                draw(data.rssi, data.step, data.active);
                // Если сканирование еще идет, запрашиваем данные каждые 5 сек
                if (data.active) {
                    setTimeout(updateChart, 5000);
                }
            });
    }

    function draw(values, currentStep, active) {
        const w = canvas.width;
        const h = canvas.height;
        ctx.clearRect(0, 0, w, h);

        // Рисуем сетку (шкала RSSI -120...-30)
        ctx.strokeStyle = '#e0e0e0';
        ctx.beginPath();
        for(let i=0; i<=4; i++) {
            let y = h - (i * h / 4);
            ctx.moveTo(0, y); ctx.lineTo(w, y);
        }
        ctx.stroke();

        // Рисуем график
        ctx.beginPath();
        ctx.lineWidth = 3;
        ctx.strokeStyle = active ? '#2196F3' : '#4CAF50'; // Синий в процессе, зеленый
    }
}

```

- финиш

```
const stepX = w / (values.length - 1);
values.forEach((v, i) => {
    // Нормализуем RSSI (-120...-30) в координаты Y (0...h)
    let val = Math.max(-120, Math.min(-30, v));
    let y = h - ((val + 120) * (h / 90));
    let x = i * stepX;

    if (i === 0) ctx.moveTo(x, y);
    else ctx.lineTo(x, y);

    // Подсвечиваем текущий шаг сканирования
    if (i === currentStep && active) {
        ctx.fillStyle = 'red';
        ctx.beginPath();
        ctx.arc(x, y, 5, 0, Math.PI * 2);
        ctx.fill();
    }
});
ctx.stroke();

// Текст статуса
ctx.fillStyle = '#333';
ctx.font = '12px Arial';
ctx.fillText(active ? `Сканирование... шаг ${currentStep}/21` : "АЧХ
построена", 10, 20);
}
```

Как это использовать на странице:

Добавьте в HTML-код страницы:

```
<canvas id="spectrumCanvas" width="600" height="200" style="border:1px solid #ddd;
width:100%"></canvas>
<button onclick="initSpectrumScanner()">Найти оптимальную частоту</button>
```

Инструкция по работе:

- Сканер будет работать медленно (около 18 минут), так как он **гарантированно** ждет пакет от датчика на каждой частоте.
- На графике вы увидите «полку» шума на уровне -108...-110 дБм и **острый пик** на частоте передатчика.

- После завершения скетч сам переключится на частоту пика.

Хотите, чтобы я оформил это в виде **готового HTML-блока со стилями**, который можно просто вставить в конец вашего файла `index.html`?

да, я использую у себя визуальный стиль mushroom

Для интеграции в стиль **Mushroom** (который часто используется в Home Assistant и веб-панелях на базе CSS-переменных), я подготовил компактный HTML-блок. Он использует скругления, тени и шрифты, характерные для этого стиля.

Добавьте этот код в ваш `index.html` (например, перед закрывающим тегом `</body>`).

```
<!-- Mushroom Style Spectrum Card -->
<div class="mushroom-card">
  <div class="mushroom-state-item">
    <div class="mushroom-icon" style="background-color: rgba(33, 150, 243, 0.1);">
      <svg style="width:24px;height:24px" viewBox="0 0 24 24">
        <path fill="#2196F3"
d="M16,15H22V17H16V15M16,10H22V12H16V10M16,19H22V21H16V19M2,15H8V17H2V15M2,19H8V21H2V19
M2,10H8V12H2V10M11,6H13V21H11V6M11,2H13V4H11V2Z" />
      </svg>
    </div>
    <div class="mushroom-info">
      <span class="mushroom-title">Анализатор частот WS1080</span>
      <span id="scanStatus" class="mushroom-subtitle">Диапазон 914.8 - 915.2
MHz</span>
    </div>
  </div>
  <div class="mushroom-content">
    <canvas id="spectrumCanvas" height="140"></canvas>
    <div class="mushroom-actions">
      <button onclick="initSpectrumScanner()" class="mushroom-button">
        Найти лучший сигнал
      </button>
    </div>
  </div>
</div>

<style>
```



```

:root {
  --msh-card-bg: #ffffff;
  --msh-title: #212121;
  --msh-subtitle: #727272;
  --msh-shadow: 0px 2px 4px 0px rgba(0,0,0,0.1);
  --msh-radius: 12px;
}

.mushroom-card {
  background: var(--msh-card-bg);
  border-radius: var(--msh-radius);
  box-shadow: var(--msh-shadow);
  padding: 16px;
  margin: 10px;
  font-family: sans-serif;
}

.mushroom-state-item {
  display: flex;
  align-items: center;
  margin-bottom: 12px;
}

.mushroom-icon {
  width: 40px;
  height: 40px;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  margin-right: 12px;
}

.mushroom-info { display: flex; flex-direction: column; }
.mushroom-title { font-weight: bold; font-size: 16px; color: var(--msh-title); }
.mushroom-subtitle { font-size: 14px; color: var(--msh-subtitle); }
.mushroom-content { width: 100%; }
#spectrumCanvas { width: 100%; border-radius: 8px; margin: 10px 0; background:
#fafafa; }

.mushroom-actions { display: flex; justify-content: flex-end; }
.mushroom-button {
  background: rgba(33, 150, 243, 0.1);
  color: #2196F3;
  border: none;
  padding: 8px 16px;
  border-radius: 8px;
  font-weight: bold;
  cursor: pointer;
}

```

```
    transition: 0.3s;
  }
  .mushroom-button:hover { background: rgba(33, 150, 243, 0.2); }
</style>
```

```
<script>
```

```
function initSpectrumScanner() {
  const statusText = document.getElementById('scanStatus');
  const canvas = document.getElementById('spectrumCanvas');
  const ctx = canvas.getContext('2d');

  // Подстраиваем внутреннее разрешение канваса под CSS
  canvas.width = canvas.offsetWidth;

  fetch('/api/start_scan').then(() => {
    statusText.innerText = "Сканирование запущено...";
    updateChart();
  });

  function updateChart() {
    fetch('/api/scan_data').then(r => r.json()).then(data => {
      draw(data.rssi, data.step, data.active);
      if (data.active) {
        statusText.innerText = `Порог: ${Math.round((data.step/21)*100)}% (${
          914.8 + data.step*0.02).toFixed(2)} MHz`;
        setTimeout(updateChart, 5000);
      } else {
        statusText.innerText = "Анализ завершен. Частота оптимизирована.";
      }
    });
  }

  function draw(values, currentStep, active) {
    const w = canvas.width, h = canvas.height;
    ctx.clearRect(0, 0, w, h);

    // Линия сетки (Noise Floor)
    ctx.setLineDash([5, 5]);
    ctx.strokeStyle = "#ddd";
    ctx.beginPath();
    ctx.moveTo(0, h * 0.8); ctx.lineTo(w, h * 0.8);
    ctx.stroke();
    ctx.setLineDash([]);
  }
}
```

```

// Рисование АЧХ
ctx.beginPath();
ctx.lineWidth = 3;
ctx.lineJoin = "round";
ctx.strokeStyle = active ? "#2196F3" : "#4CAF50";

const stepX = w / (values.length - 1);
values.forEach((v, i) => {
    let val = Math.max(-120, Math.min(-30, v));
    let y = h - ((val + 120) * (h / 90));
    let x = i * stepX;
    if (i === 0) ctx.moveTo(x, y); else ctx.lineTo(x, y);

    if (i === currentStep && active) {
        ctx.fillStyle = "#F44336";
        ctx.beginPath(); ctx.arc(x, y, 4, 0, Math.PI*2); ctx.fill();
    }
});
ctx.stroke();
}
}
</script>

```

Как пользоваться этой панелью:

1. **Нажали кнопку:** скетч на ESP32 начинает 18-минутный цикл.
2. **Визуализация:** на графике в реальном времени будут появляться точки.
3. **Результат:** если датчик передал данные, вы увидите характерный пик. Зеленый цвет графика в конце означает, что сканер завершил работу и применил лучшую частоту.

Обратите внимание: в C++ коде сканера я поставил шаг **0.02 (20 кГц)**. Это идеальный компромисс для WS1080: достаточно точно, чтобы попасть в пик, и не слишком долго.

Вам помочь с **интеграцией этого API** в вашу основную таблицу обработчиков `server.on` в скетче?

да, в качестве примера очень интересно.

Чтобы функционал АЧХ-сканера заработал, его нужно «прописать» в основной цикл программы и добавить новые маршруты (API) в секцию настройки сервера.

Вот как выглядит интеграция в ваш основной скетч на примере ключевых блоков:

1. Добавление в `loop()`

Чтобы сканер мог переключать частоты и замерять RSSI в фоновом режиме (не мешая Web-серверу и приему данных), добавьте вызов функции в самый верх цикла:

```
void loop() {
  server.handleClient();

  // Вызов обработчика сканера
  updateAutoScanner();

  // Основная работа радио (прием пакетов)
  // Работает только когда scan_active == false, чтобы не мешать сканированию
  if (!scan_active) {
    rf.loop();
  }

  yield(); // Даем время фоновым процессам
}
```

2. Регистрация API в `setup()`

Добавьте эти два обработчика в секцию, где вы настраиваете `server.on()`. Это позволит JavaScript-коду с веб-страницы управлять сканером.

```
// --- Секция в setup() ---

// 1. Команда на запуск сканирования
server.on("/api/start_scan", HTTP_GET, []() {
  scan_active = true;
  scan_current_step = 0;
  scan_best_rssi = -150.0;
  // Очищаем старые результаты перед новым запуском
  for(int i = 0; i < 21; i++) scan_results[i] = -120.0;

  LOG("SCAN", "User started frequency analysis");
  server.send(200, "text/plain", "OK");
});

// 2. Отдача данных для отрисовки графика
```

```

server.on("/api/scan_data", HTTP_GET, []() {
    JsonDocument doc;
    doc["active"] = scan_active;
    doc["step"] = scan_current_step;

    // Массив RSSI для графика
    JsonArray data = doc.createNestedArray("rssi");
    for (int i = 0; i < 21; i++) {
        data.add(scan_results[i]);
    }

    String out;
    serializeJson(doc, out);
    server.send(200, "application/json", out);
});

```

3. Нюанс с функцией `updateAutoScanner()`

Убедитесь, что внутри функции в конце цикла сканирования стоит сохранение частоты, если результат нас устроил:

```

if (scan_current_step >= 21) {
    scan_active = false;
    // Если нашли сигнал сильнее -95dBm (значит это не шум, а датчик)
    if (scan_best_rssi > -95.0) {
        current_freq = scan_best_freq;
        radio.setFrequency(current_freq);
        saveAll(); // Ваша функция сохранения настроек в Preferences
        LOG("SCAN", "Auto-tuning complete: Set to %.2f MHz", current_freq);
    }
}

```

Что вы увидите в итоге:

1. При нажатии кнопки «Найти лучший сигнал» на веб-странице, ESP32 перейдет в режим сканирования.
2. Светодиод `RF_RX` перестанет моргать на 18 минут (так как `rf.loop()` приостановлен), но зато в логе и на графике будут появляться точки уровня сигнала.
3. Как только датчик «крикнет» на какой-то частоте, на графике в браузере вырастет пик.
4. По окончании станция сама вернется в режим приема, но уже на идеально подстроенной частоте.

Хотите добавить на график горизонтальную линию «Порог шума», чтобы визуально было понятно, где сигнал датчика, а где просто радиомусор?

да, это увеличит информативность.

Чтобы добавить линию порога шума (Noise Floor), нам нужно передать среднее значение фонового шума из контроллера и отобразить его на графике. Это позволит сразу отличить полезный сигнал от эфирного «мусора».

1. Изменения в C++ (Back-end)

Добавим расчет среднего шума в API. Обычно шум для CC1101 на 915 МГц составляет около -108...-110 дБм.

В обработчик `server.on("/api/scan_data", ...)` добавьте одну строку:

```
server.on("/api/scan_data", HTTP_GET, []() {
    JsonDocument doc;
    doc["active"] = scan_active;
    doc["step"] = scan_current_step;
    doc["noise"] = -108.5; // Типовой порог шума для CC1101

    JsonArray data = doc.createNestedArray("rssi");
    for (int i = 0; i < 21; i++) data.add(scan_results[i]);

    String out;
    serializeJson(doc, out);
    server.send(200, "application/json", out);
});
```

2. Изменения в JavaScript (Front-end)

Обновим функцию `draw`, чтобы она рисовала пунктирную линию и подпись.

```
function draw(values, currentStep, active, noiseFloor = -108.5) {
    const w = canvas.width, h = canvas.height;
    ctx.clearRect(0, 0, w, h);

    // --- РИСУЕМ ПОРОГ ШУМА ---
    const noiseY = h - ((noiseFloor + 120) * (h / 90));
    ctx.setLineDash([5, 5]); // Пунктир
    ctx.strokeStyle = "rgba(244, 67, 54, 0.4)"; // Полупрозрачный красный
    ctx.lineWidth = 1;
    ctx.beginPath();
```

```

    ctx.moveTo(0, noiseY);
    ctx.lineTo(w, noiseY);
    ctx.stroke();

    ctx.fillStyle = "rgba(244, 67, 54, 0.6)";
    ctx.font = "10px sans-serif";
    ctx.fillText("ШУМ", 5, noiseY - 5);
    ctx.setLineDash([]); // Возвращаем сплошную линию

    // --- РИСУЕМ ГРАФИК АЧХ ---
    ctx.beginPath();
    ctx.lineWidth = 3;
    ctx.lineJoin = "round";
    ctx.strokeStyle = active ? "#2196F3" : "#4CAF50";

    const stepX = w / (values.length - 1);
    values.forEach((v, i) => {
        let val = Math.max(-120, Math.min(-30, v));
        let y = h - ((val + 120) * (h / 90));
        let x = i * stepX;

        if (i === 0) ctx.moveTo(x, y);
        else ctx.lineTo(x, y);

        // Точка текущего прогресса
        if (i === currentStep && active) {
            ctx.fillStyle = "#2196F3";
            ctx.beginPath(); ctx.arc(x, y, 4, 0, Math.PI*2); ctx.fill();
        }
    });
    ctx.stroke();
}

```

Что это даст визуально:

1. **Красная пунктирная линия:** Будет идти горизонтально в нижней части графика.
2. **Полезный сигнал:** Вы увидите, как на определенной частоте график «выстреливает» далеко вверх над этой линией. Если пик едва касается линии шума — значит, антенна настроена плохо или датчик слишком далеко.
3. **Информативность:** Теперь вы точно знаете, что всё, что ниже этой линии — пустой эфир, а всё, что выше — работа вашей станции.

Совет по калибровке:

Если ваш «горб» на графике смещен относительно центра (915.00), попробуйте вручную выставить найденную частоту в настройках. Часто после такой подстройки удается поймать сигнал даже там, где раньше «монитор» станции показывал отсутствие связи.

Нужно ли добавить на график **метки частот** (например, 914.8, 915.0, 915.2) под нижней осью для удобства?

думается сделать может быть просто в виде черточек небольших по горизонтальной оси, чтобы сильно не загромождать интерфейс. или очень маленькими цифирками.

Это правильный подход: лишние цифры перегружают «чистый» стиль Mushroom. Тонкие засечки (черточки) и мелкие подписи по краям и в центре создадут ощущение профессионального прибора, не нарушая эстетику.

Вот обновленный блок отрисовки оси частот для вашей функции `draw`:

Обновленный JavaScript (секция отрисовки оси)

Добавьте этот фрагмент в конец функции `draw`, после того как отрисован основной график:

```
// --- РИСУЕМ ОСЬ ЧАСТОТ (X-axis) ---
ctx.fillStyle = "#9e9e9e"; // Светло-серый цвет для подписей
ctx.font = "9px sans-serif";
ctx.textAlign = "center";

const freqs = ["914.8", "915.0", "915.2"]; // Ключевые точки
const labelY = h - 5; // Позиция подписей чуть выше нижнего края

freqs.forEach((label, i) => {
  // Вычисляем X для начала, середины и конца
  const x = (i * (w / 2));

  // Рисуем маленькую засечку
  ctx.strokeStyle = "#bdbdbd";
  ctx.beginPath();
  ctx.moveTo(x, h);
  ctx.lineTo(x, h - 8);
  ctx.stroke();
```



```

// Рисуем цифру (с небольшим смещением для краев)
let alignX = x;
if (i === 0) alignX = x + 15;
if (i === 2) alignX = x - 15;
ctx.fillText(label + " MHz", alignX, labelY);
});

// Добавим мелкие промежуточные засечки без цифр
ctx.beginPath();
for (let i = 1; i < 20; i++) {
  if (i === 10) continue; // Пропускаем центр (там уже есть большая)
  const x = i * (w / 20);
  ctx.moveTo(x, h);
  ctx.lineTo(x, h - 4);
}
ctx.stroke();

```

Как это будет выглядеть в стиле Mushroom:

1. **Засечки:** Внизу графика появятся аккуратные «деления», как на аналоговой шкале радиоприемника.
2. **Минимализм:** Мы подписали только начало, середину и конец диапазона. Этого достаточно, чтобы понять, в какой стороне находится «горб» АЧХ.
3. **Акцент на данных:** Основное внимание остается на линии графика и красной линии шума.

Маленький совет по коду ESP32:

Чтобы график не «дергался» при каждом обновлении, в API `scan_data` можно добавить небольшое сглаживание или отдавать `null` для тех шагов, которые еще не пройдены. Тогда JavaScript будет рисовать линию только до текущего момента сканирования.

Теперь ваш анализатор выглядит как законченное инженерное решение. **Желаете ли вы добавить кнопку «Применить сейчас»**, чтобы можно было вручную выбрать любую точку на графике и настроиться на неё кликом мышки?

гм, думаю это интересное предложение. делаем.

Это превратит АЧХ-сканер в полноценный интерактивный инструмент: вы сможете кликнуть на любой «пик» на графике, и станция мгновенно перестроится на эту частоту.

Для этого нам нужно добавить обработчик клика на Canvas и новый API-метод.

1. Обработчик клика в JavaScript

Добавьте этот код в ваш тег `<script>`. Он переводит координаты клика в значение частоты.

```
// Добавьте это внутрь или после функции initSpectrumScanner
document.getElementById('spectrumCanvas').onclick = function(e) {
  const canvas = e.target;
  const rect = canvas.getBoundingClientRect();
  const x = e.clientX - rect.left; // Клик по горизонтали
  const w = canvas.offsetWidth;

  // Вычисляем частоту: начало 914.80 + (процент клика от ширины * диапазон 0.4)
  const clickedFreq = (914.80 + (x / w) * 0.4).toFixed(2);

  if (confirm(`Настроиться на частоту ${clickedFreq} MHz?`)) {
    setFrequency(clickedFreq);
  }
};

function setFrequency(f) {
  fetch(`/api/set_freq?f=${f}`, { method: 'POST' })
    .then(r => {
      if(r.ok) {
        alert(`Частота ${f} MHz установлена!`);
        // Обновляем заголовок на странице, если нужно
        if(document.getElementById('currentFreqDisplay')) {
          document.getElementById('currentFreqDisplay').innerText = f;
        }
      }
    });
}
```

2. Изменения в C++ (API)

Убедитесь, что ваш обработчик `/api/set_freq` в скетче понимает простую установку частоты через `radio.setFrequency()`.

```
server.on("/api/set_freq", HTTP_POST, []() {
  if (server.hasArg("f")) {
    float newFreq = server.arg("f").toFloat();
```

```

// 1. Устанавливаем в чип
radio.setFrequency(newFreq);

// 2. Обновляем глобальную переменную и сохраняем
current_freq = newFreq;
saveAll();

LOG("API", "Frequency manually set to %.2f MHz", current_freq);
server.send(200, "text/plain", "OK");
} else {
server.send(400, "text/plain", "Missing frequency");
}
});

```

Как это работает в жизни:

1. Вы запускаете сканирование и видите «горб» сигнала, например, чуть правее центра.
2. Вы **просто кликаете мышкой (или пальцем на телефоне)** прямо по этому горбу на графике.
3. Браузер высчитывает, что вы кликнули, например, на 75% ширины графика, что соответствует **915.10 MHz**.
4. Станция тут же перестраивается, сохраняет настройки, и вы начинаете получать пакеты с максимальным уровнем сигнала.

Визуальный совет:

Чтобы пользователь понимал, что на график можно кликать, добавьте в CSS для канваса одну строчку:

```
#spectrumCanvas { cursor: crosshair; }
```

Теперь ваш проект — это не просто «слушатель» погодной станции, а настоящий **радиосканер с визуальным управлением**.

Нужно ли добавить на график вертикальную «нитку» (маркер), которая будет показывать текущую настроенную частоту относительно всего диапазона? Это поможет видеть, насколько текущая настройка совпадает с реальным пиком.

Вертикальная «нитка» (маркер) — это финальный штрих. Она будет наглядно показывать, где сейчас «находится» приемник относительно спектра. Если маркер

стоит на 915.00, а «горб» сигнала на 915.08, вы сразу увидите необходимость подстройки.

1. Доработка JavaScript (Функция отрисовки маркера)

Добавьте этот блок в функцию `draw` (в самый конец, чтобы линия была поверх графика):

```
// --- РИСУЕМ МАРКЕР ТЕКУЩЕЙ ЧАСТОТЫ (Нитка) ---
// Предположим, глобальная переменная current_freq_val хранит текущую частоту (напр.
915.00)
// Её можно брать из данных /api/status или передавать в /api/scan_data

const currentFreq = window.current_freq_val || 915.00;
const markerX = ((currentFreq - 914.80) / 0.4) * w;

if (markerX >= 0 && markerX <= w) {
  ctx.setLineDash([5, 3]); // Пунктир
  ctx.strokeStyle = "#FF9800"; // Оранжевый (цвет Mushroom для активных элементов)
  ctx.lineWidth = 2;

  ctx.beginPath();
  ctx.moveTo(markerX, 0);
  ctx.lineTo(markerX, h);
  ctx.stroke();

  // Флажок с частотой наверху
  ctx.setLineDash([]);
  ctx.fillStyle = "#FF9800";
  ctx.beginPath();
  ctx.arc(markerX, 0, 4, 0, Math.PI * 2);
  ctx.fill();
}
```

2. Подсказка для пользователя (CSS)

Чтобы интерфейс стал еще отзывчивее, добавьте эффект наведения на Canvas. Это подчеркнет интерактивность:

```
#spectrumCanvas {
  cursor: crosshair;
```

```

    transition: background 0.2s;
}
#spectrumCanvas:hover {
    background: #f0f7ff !important; /* Легкая подсветка при наведении */
}

```

3. Как обновлять «нитку»

Чтобы маркер перемещался сразу после клика, обновите функцию `setFrequency`:

```

function setFrequency(f) {
    fetch(`/api/set_freq?f=${f}`, { method: 'POST' })
    .then(r => {
        if(r.ok) {
            window.current_freq_val = parseFloat(f); // Обновляем положение нитки
            // Перерисовываем график, если он уже есть
            if (window.lastScanData) {
                draw(window.lastScanData.rssi, window.lastScanData.step,
window.lastScanData.active);
            }
        }
    });
}

```

Итоговый сценарий использования:

1. Вы открываете страницу и видите график АЧХ (после сканирования).
2. Оранжевая пунктирная «нитка» стоит на **915.00**.
3. Вы видите, что реальный пик сигнала от погодной станции находится чуть правее.
4. Вы **кликаете на пик**.
5. «Нитка» мгновенно перепрыгивает на место клика, станция перенастраивается, и вы видите, как данные в таблице начинают обновляться без пропусков.

Теперь ваша система управления радиоканалом выглядит максимально профессионально и интуитивно.

Есть ли еще какие-то параметры погодной станции, которые вы хотели бы визуализировать (например, график заряда батареи внешнего блока или историю силы сигнала)?

да

Вертикальная «нитка» (маркер) — это финальный штрих. Она будет наглядно показывать, где сейчас «находится» приемник относительно спектра. Если маркер стоит на 915.00, а «горб» сигнала на 915.08, вы сразу увидите необходимость подстройки.

1. Доработка JavaScript (Функция отрисовки маркера)

Добавьте этот блок в функцию `draw` (в самый конец, чтобы линия была поверх графика):

```
// --- РИСУЕМ МАРКЕР ТЕКУЩЕЙ ЧАСТОТЫ (Нитка) ---
// Предположим, глобальная переменная current_freq_val хранит текущую частоту (напр.
915.00)
// Её можно брать из данных /api/status или передавать в /api/scan_data

const currentFreq = window.current_freq_val || 915.00;
const markerX = ((currentFreq - 914.80) / 0.4) * w;

if (markerX >= 0 && markerX <= w) {
  ctx.setLineDash([5, 3]); // Пунктир
  ctx.strokeStyle = "#FF9800"; // Оранжевый (цвет Mushroom для активных элементов)
  ctx.lineWidth = 2;

  ctx.beginPath();
  ctx.moveTo(markerX, 0);
  ctx.lineTo(markerX, h);
  ctx.stroke();

  // Флажок с частотой наверху
  ctx.setLineDash([]);
  ctx.fillStyle = "#FF9800";
  ctx.beginPath();
  ctx.arc(markerX, 0, 4, 0, Math.PI * 2);
  ctx.fill();
}
```

2. Подсказка для пользователя (CSS)

Чтобы интерфейс стал еще отзывчивее, добавьте эффект наведения на Canvas. Это подчеркнет интерактивность:

```
#spectrumCanvas {
  cursor: crosshair;
  transition: background 0.2s;
```

```
}  
#spectrumCanvas: hover {  
  background: #f0f7ff !important; /* Легкая подсветка при наведении */  
}
```

3. Как обновлять «нитку»

Чтобы маркер перемещался сразу после клика, обновите функцию `setFrequency`:

```
function setFrequency(f) {  
  fetch(`/api/set_freq?f=${f}`, { method: 'POST' })  
    .then(r => {  
      if(r.ok) {  
        window.current_freq_val = parseFloat(f); // Обновляем положение нитки  
        // Перерисовываем график, если он уже есть  
        if (window.lastScanData) {  
          draw(window.lastScanData.rssi, window.lastScanData.step,  
window.lastScanData.active);  
        }  
      }  
    });  
}
```

Итоговый сценарий использования:

1. Вы открываете страницу и видите график АЧХ (после сканирования).
2. Оранжевая пунктирная «нитка» стоит на **915.00**.
3. Вы видите, что реальный пик сигнала от погодной станции находится чуть правее.
4. Вы **кликаете на пик**.
5. «Нитка» мгновенно перепрыгивает на место клика, станция перенастраивается, и вы видите, как данные в таблице начинают обновляться без пропусков.

Теперь ваша система управления радиоканалом выглядит максимально профессионально и интуитивно.

Есть ли еще какие-то параметры погодной станции, которые вы хотели бы визуализировать (например, график заряда батареи внешнего блока или историю силы сигнала)?